

BugHound

Error Knowledge Base

32 recurring error patterns across Python, FastAPI, LangChain, JavaScript/Node.js, Docker, and deployment. Each entry is a generic, previously-seen pattern — the running system matches new errors against this document instead of calling a model when a strong match exists.

Framework	Entries	Count
Docker	E027, E028, E029, E030	4
FastAPI	E011, E012, E013, E014, E015	5
General	E001, E002, E003, E004, E005, E006, E007, E008, E009, E010, E023, E025	12
Git	E032	1
LangChain	E016, E017, E018, E019, E020	5
Node.js	E021, E022, E026	3
Railway	E031	1
npm	E024	1

=== ENTRY E001 ===

ModuleNotFoundError after a package restructure

Language: Python | Framework: General | Tags: import, packaging, module-not-found | Confidence: 0.88

Error Pattern:

ModuleNotFoundError: No module named

Root Cause:

The import path points at a module that a newer version of the package moved, renamed, or removed. This is common right after upgrading a fast-moving library — the old top-level module becomes a thin shim for a few releases, then disappears entirely.

Fix:

Check the package's changelog for the release that introduced the restructure and update the import to the new location. If the package was split into sub-packages, install the specific sub-package.

Patch Template:

```
--- a/app/main.py
+++ b/app/main.py
@@
-from langchain.chat_models import ChatOpenAI
+from langchain_openai import ChatOpenAI
```

=== END E001 ===

=== ENTRY E002 ===

ImportError: cannot import name X from partially initialized module

Language: Python | Framework: General | Tags: import, circular-import | Confidence: 0.75

Error Pattern:

ImportError: cannot import name

Root Cause:

Two modules import each other at module load time (a circular import). Python starts executing module A, which imports module B, which tries to import a name from A before A has finished defining it.

Fix:

Move the shared import to function scope (import inside the function that needs it), or extract the shared symbol into a third module that both A and B import from, breaking the cycle.

Patch Template:

```
--- a/app/services/a.py
+++ b/app/services/a.py
@@
-from app.services.b import helper

def run():
+    from app.services.b import helper
    return helper()
```

=== END E002 ===

=== ENTRY E003 ===

TypeError: missing N required positional argument

Language: Python | Framework: General | Tags: `TypeError`, `function-signature` | Confidence: 0.70

Error Pattern:

`TypeError: .* missing .* required positional argument`

Root Cause:

A function or method is being called with fewer arguments than its signature requires. Frequently happens after adding a new parameter without a default, or forgetting ``self`` on an instance method mistakenly called as if it were a static function.

Fix:

Pass the missing argument at the call site, or give the parameter a default value if it is genuinely optional.

=== END E003 ===

=== ENTRY E004 ===

KeyError on a dictionary access

Language: Python | Framework: General | Tags: `KeyError`, `dict` | Confidence: 0.72

Error Pattern:

`KeyError:`

Root Cause:

Code indexes a dictionary with ``d[key]`` for a key that is not guaranteed to exist — typically a field missing from an API payload, a config file, or an environment mapping.

Fix:

Use ``d.get(key, default)`` when a missing key is expected, or validate the input shape up front (a Pydantic model or an explicit check) so the failure is caught with a clear message before it reaches this line.

Patch Template:

```
--- a/app/handlers.py
+++ b/app/handlers.py
@@
-user_id = payload['user_id']
+user_id = payload.get('user_id')
+if user_id is None:
+    raise ValueError("payload is missing 'user_id'")
```

=== END E004 ===

=== ENTRY E005 ===

AttributeError: NoneType has no attribute X

Language: Python | Framework: General | Tags: attributeerror, none | Confidence: 0.68

Error Pattern:

AttributeError: 'NoneType' object has no attribute

Root Cause:

A function that can return `None` — a dict `.get()`, a failed lookup, an ORM query with no match — is being used as if it always returns an object, and the `None` case was never handled.

Fix:

Check for `None` immediately after the call that can produce it, and either return early, raise a clear error, or supply a default.

=== END E005 ===

=== ENTRY E006 ===

RecursionError: maximum recursion depth exceeded

Language: Python | Framework: General | Tags: recursion, stack | Confidence: 0.65

Error Pattern:

RecursionError: maximum recursion depth exceeded

Root Cause:

A recursive function has no base case that is ever reached for the given input, or two functions call each other indefinitely (mutual recursion without a terminating condition).

Fix:

Add or fix the base case so recursion terminates, or convert the function to an iterative loop with an explicit stack if the recursion depth is expected to be legitimately large.

=== END E006 ===

=== ENTRY E007 ===

UnicodeDecodeError while reading a file

Language: Python | Framework: General | Tags: encoding, unicode, file-io | Confidence: 0.70

Error Pattern:

UnicodeDecodeError: 'utf-8' codec can't decode byte

Root Cause:

A file is opened with the default or an assumed UTF-8 encoding, but its actual bytes are in a different encoding (commonly Latin-1/cp1252 on files that came from Windows tools).

Fix:

Open the file with the correct encoding, or detect it (e.g. with ``charset-normalizer``) instead of assuming UTF-8. As a last resort, decode with ``errors="replace"`` if some data loss is acceptable.

Patch Template:

```
--- a/app/importer.py
+++ b/app/importer.py
@@
- with open(path) as f:
+ with open(path, encoding="utf-8-sig") as f:
```

=== END E007 ===

=== ENTRY E008 ===

IndentationError: unexpected indent

Language: Python | Framework: General | Tags: syntax, indentation | Confidence: 0.80

Error Pattern:

IndentationError:

Root Cause:

Tabs and spaces are mixed in the same block, or an editor auto-indented a line to a level Python does not accept relative to its enclosing block.

Fix:

Re-indent the file consistently (spaces only, 4 per level is the PEP 8 convention) and configure the editor to insert spaces for Tab.

=== END E008 ===

=== ENTRY E009 ===

ZeroDivisionError: division by zero

Language: Python | Framework: General | Tags: arithmetic, zerodivision | Confidence: 0.75

Error Pattern:

ZeroDivisionError: division by zero

Root Cause:

A denominator that is computed from input data — a count, a duration, an average's sample size — reaches zero for an edge case, such as an empty list, that the calling code did not anticipate.

Fix:

Guard the division with a check for a zero denominator and return a sensible default (0, None, or skip the calculation) for that case.

=== END E009 ===

=== ENTRY E010 ===

json.decoder.JSONDecodeError: Expecting value

Language: Python | Framework: General | Tags: json, parsing | Confidence: 0.70

Error Pattern:

json.decoder.JSONDecodeError: Expecting value

Root Cause:

`json.loads()` is called on a string that is empty, is HTML (often an error page from an API that returned a non-JSON response), or is `None` coerced to the literal text 'None'.

Fix:

Check the HTTP status code and content-type before parsing, and log the raw response body when parsing fails so the real cause is visible.

=== END E010 ===

=== ENTRY E011 ===

FastAPI 422 Unprocessable Entity on a POST request

Language: Python | Framework: FastAPI | Tags: validation, pydantic, 422 | Confidence: 0.85

Error Pattern:

422 Unprocessable Entity.*Field required

Root Cause:

The request body is missing a field the Pydantic model declares as required, or the client's JSON key does not match the model's attribute name (e.g. camelCase from a JS client vs snake_case in the model).

Fix:

Make the field Optional with a default if it truly is optional, add `Field(alias="...")` to match the client's key name, or fix the client payload to match the schema.

Patch Template:

```
--- a/app/models.py
+++ b/app/models.py
@@
-     user_id: str
+     user_id: str = Field(..., alias="userId")
+
+     class Config:
+         populate_by_name = True
```

=== END E011 ===

=== ENTRY E012 ===

FastAPI dependency injection: 'Depends object has no attribute'

Language: Python | Framework: FastAPI | Tags: dependency-injection, depends | Confidence: 0.78

Error Pattern:

AttributeError.*Depends

Root Cause:

A dependency function is called directly in normal Python code instead of being resolved by FastAPI's injection system, so the parameter holds the `Depends(...)` sentinel object rather than the value it produces.

Fix:

Only call the dependency-producing function through a route parameter typed as `Annotated[X, Depends(get\_x)]` (or the equivalent `x: X = Depends(get\_x)`); never call it as a plain function outside a request handler.

=== END E012 ===

=== ENTRY E013 ===

FastAPI CORS: 'No Access-Control-Allow-Origin header'

Language: Python | Framework: FastAPI | Tags: cors, middleware | Confidence: 0.90

Error Pattern:

No 'Access-Control-Allow-Origin' header

Root Cause:

`CORSMiddleware` is missing, or is configured with an `allow\_origins` list that does not include the calling frontend's exact origin (scheme + host + port must match precisely).

Fix:

Add `CORSMiddleware` with the frontend's origin in `allow\_origins`, and set `allow\_credentials=True` if the frontend sends cookies.

Patch Template:

```
--- a/app/main.py
+++ b/app/main.py
@@
+from fastapi.middleware.cors import CORSMiddleware
+
+app.add_middleware(
+    CORSMiddleware,
+    allow_origins=["http://localhost:3000"],
+    allow_methods=["*"],
+    allow_headers=["*"],
+)
```

=== END E013 ===

=== ENTRY E014 ===

Pydantic v2 migration: 'BaseSettings has been moved'

Language: Python | Framework: FastAPI | Tags: pydantic, migration, settings | Confidence: 0.92

Error Pattern:

BaseSettings.*has been moved|pydantic.errors.PydanticUserError

Root Cause:

Pydantic v2 moved `BaseSettings` out of the core package into the separate `pydantic-settings` package. Code written for Pydantic v1 imports it from `pydantic` directly, which no longer works.

Fix:

Install `pydantic-settings` and import `BaseSettings` from it instead.

Patch Template:

```
--- a/app/config.py
+++ b/app/config.py
@@
-from pydantic import BaseSettings
+from pydantic_settings import BaseSettings
```

=== END E014 ===

=== ENTRY E015 ===

FastAPI background task never seems to run

Language: Python | Framework: FastAPI | Tags: background-tasks, async | Confidence: 0.70

Error Pattern:

BackgroundTasks

Root Cause:

The `BackgroundTasks` parameter was added to the route but never had `.add\_task(...)` called on it, or the task was added to a locally created `BackgroundTasks()` instance instead of the one injected by FastAPI, so it is discarded when the response is returned.

Fix:

Accept `background\_tasks: BackgroundTasks` as a route parameter and call `background\_tasks.add\_task(fn, \*args)` on that exact instance.

=== END E015 ===

=== ENTRY E016 ===

LangChain: No module named 'langchain.chat_models'

Language: Python | Framework: LangChain | Tags: langchain, import, packaging | Confidence: 0.93

Error Pattern:

No module named 'langchain.chat_models'

Root Cause:

LangChain split provider integrations into standalone packages (`langchain-openai`, `langchain-anthropic`, etc.) during the 0.1 -> 0.2 restructure. `langchain.chat\_models.ChatOpenAI` no longer exists in current releases.

Fix:

Install the provider-specific package and import the chat model class from it directly.

Patch Template:

```
--- a/app/main.py
+++ b/app/main.py
@@
-from langchain.chat_models import ChatOpenAI
+from langchain_openai import ChatOpenAI
```

=== END E016 ===

=== ENTRY E017 ===

LangChain: LangChainDeprecationWarning on memory classes

Language: Python | Framework: LangChain | Tags: langchain, deprecation, memory | Confidence: 0.60

Error Pattern:

LangChainDeprecationWarning.*memory

Root Cause:

Classes like `ConversationBufferMemory` are deprecated in favour of LangGraph's persistence layer or explicit message-list state. The old classes still work but will be removed in a future release.

Fix:

For new code, hold conversation state as a plain list of messages, or use a LangGraph checkpointer if persistence is required. For a short-term fix, the deprecated class still functions — the warning is not yet a hard failure.

=== END E017 ===

=== ENTRY E018 ===

LangChain: OutputParserException on structured output

Language: Python | Framework: LangChain | Tags: langchain, output-parser, json | Confidence: 0.65

Error Pattern:

OutputParserException

Root Cause:

The model's raw text response could not be parsed into the expected structure — usually because a smaller or free-tier model added commentary, markdown fences, or explanatory text around the JSON instead of returning only the JSON object.

Fix:

Tighten the prompt to demand JSON only with no surrounding text, add a one-shot repair step that re-asks the model to fix invalid output, or switch to a model with reliable structured-output support.

=== END E018 ===

=== ENTRY E019 ===

LangChain: tool calling schema mismatch with a free model

Language: Python | Framework: LangChain | Tags: langchain, tool-calling, free-model | Confidence: 0.60

Error Pattern:

does not support tools|tool_choice

Root Cause:

The selected model does not support native function/tool calling, or supports a different calling convention than the one LangChain's `bind\_tools` assumes for that provider — common with smaller free-tier models on OpenRouter.

Fix:

Check the model's `supported\_parameters` on the provider's model listing for `tools`; if absent, fall back to prompt-based JSON output instead of native tool calling for that model.

=== END E019 ===

=== ENTRY E020 ===

openai.NotFoundError: 404 No endpoints found for <model>:free

Language: Python | Framework: LangChain | Tags: openrouter, free-model, 404 | Confidence: 0.85

Error Pattern:

No endpoints found for.*:free|404.*model

Root Cause:

The configured free model ID has been removed or renamed by its provider on OpenRouter. Free-tier model availability rotates without notice; an ID that worked last month can disappear entirely.

Fix:

Query OpenRouter's public model catalog for models currently priced at zero and switch to one of them; do not hard-code a single free model ID as a permanent default.

=== END E020 ===

=== ENTRY E021 ===

Node.js: Cannot find module 'X'

Language: JavaScript | Framework: Node.js | Tags: node, module-not-found, npm | Confidence: 0.80

Error Pattern:

Error: Cannot find module

Root Cause:

The package is listed in `package.json` but was never installed (no `node\_modules` entry), or the import path has a typo, or the package was installed as a devDependency but is required at runtime in production where devDependencies are pruned.

Fix:

Run `npm install`, verify the exact package name and path, and move the dependency out of devDependencies if it is needed at runtime.

=== END E021 ===

=== ENTRY E022 ===

Node.js: ERR_REQUIRE_ESM importing an ES Module with require()

Language: JavaScript | Framework: Node.js | Tags: esm, commonjs, require | Confidence: 0.78

Error Pattern:

ERR_REQUIRE_ESM

Root Cause:

The target package ships only as an ES Module (`"type": "module"` or `.mjs`), but the calling code uses CommonJS `require()`, which cannot load ESM synchronously.

Fix:

Switch the calling file to ESM (`import`) and set `"type": "module"` in `package.json`, or use a dynamic `await import(...)` from CommonJS code.

Patch Template:

```
--- a/server.js
+++ b/server.js
@@
-const chalk = require('chalk');
+const { default: chalk } = await import('chalk');
```

=== END E022 ===

=== ENTRY E023 ===

TypeError: X is not a function

Language: JavaScript | Framework: General | Tags: `typescript`, `undefined` | Confidence: 0.62

Error Pattern:

TypeError: `.*` is not a function

Root Cause:

A value that used to be a function in an older version of a library is now an object, or a default export is being called as a named export (or vice versa), which is common after a major version bump.

Fix:

Check the library's current changelog for the exact call signature and update the import/usage to match; log ``typeof x`` at the call site while debugging to confirm what the value actually is.

=== END E023 ===

=== ENTRY E024 ===

npm ERESOLVE: unable to resolve dependency tree

Language: JavaScript | Framework: npm | Tags: `npm`, `peer-dependency`, `eresolve` | Confidence: 0.68

Error Pattern:

ERESOLVE unable to resolve dependency tree

Root Cause:

Two installed packages require incompatible versions of the same peer dependency (commonly React or a shared framework), and npm 7+ enforces peer dependency resolution strictly by default.

Fix:

Upgrade the outdated package to a version compatible with the shared peer dependency; ``--legacy-peer-deps`` or ``--force`` unblock the install but leave the underlying incompatibility unresolved.

=== END E024 ===

=== ENTRY E025 ===

Fetch/CORS: 'has been blocked by CORS policy' in the browser console

Language: JavaScript | Framework: General | Tags: cors, fetch, browser | Confidence: 0.82

Error Pattern:

has been blocked by CORS policy

Root Cause:

The browser blocked the response because the server did not send an `Access-Control-Allow-Origin` header matching the page's origin. This is enforced by the browser, not the server, so the request often succeeds when tested from curl or Postman, which is a common source of confusion.

Fix:

Configure CORS on the server to allow the frontend's exact origin; a browser-side workaround (a proxy, disabling security) is not a real fix for anything beyond local development.

=== END E025 ===

=== ENTRY E026 ===

UnhandledPromiseRejectionWarning / unhandled promise rejection

Language: JavaScript | Framework: Node.js | Tags: promise, async, error-handling | Confidence: 0.72

Error Pattern:

UnhandledPromiseRejectionWarning|unhandled promise rejection

Root Cause:

An `async` function throws or a Promise rejects, and nothing in the call chain has a `.catch()` or a surrounding `try/catch` on the `await`.

Fix:

Wrap the `await` in `try/catch` at the boundary where the error should be handled (a route handler, a job runner), and add a process-level `unhandledRejection` listener as a safety net, not as the primary fix.

=== END E026 ===

=== ENTRY E027 ===

Docker: 'permission denied' writing inside the container

Language: General | Framework: Docker | Tags: docker, permissions, non-root | Confidence: 0.75

Error Pattern:

PermissionError|EACCES.*permission denied

Root Cause:

The container runs as a non-root user (a good security practice) but the application tries to write to a directory still owned by root from the image build step, such as a log or upload folder created before the `USER` instruction switched accounts.

Fix:

`chown` the specific writable directories to the non-root user before the `USER` instruction in the Dockerfile, or create them as that user from the start.

Patch Template:

```
--- a/Dockerfile
+++ b/Dockerfile
@@
+RUN mkdir -p /app/logs && chown -R appuser:appuser /app/logs
+USER appuser
```

=== END E027 ===

=== ENTRY E028 ===

Docker: 'port is already allocated'

Language: General | Framework: Docker | Tags: docker, networking, port | Confidence: 0.85

Error Pattern:

port is already allocated|address already in use

Root Cause:

Another container, or a process on the host, is already bound to the port this container is trying to publish — frequently a previous run of the same container that was not removed.

Fix:

Stop the conflicting container (`docker ps` then `docker stop`), or publish on a different host port with `-p 8001:8000`.

=== END E028 ===

=== ENTRY E029 ===

Docker: 'exec format error' running a built image

Language: General | Framework: Docker | Tags: docker, architecture, arm, amd64 | Confidence: 0.80

Error Pattern:

exec.*exec format error

Root Cause:

The image was built for a different CPU architecture than the host is running — typically an amd64 image built on an Apple Silicon (arm64) machine without cross-platform build flags, then run on an amd64 server or vice versa.

Fix:

Build with ``docker buildx build --platform linux/amd64,linux/arm64`` and push a multi-arch manifest, or specify the correct target platform explicitly at build and run time.

=== END E029 ===

=== ENTRY E030 ===

Docker Compose: 'service healthcheck failed' loop / restart loop

Language: General | Framework: Docker | Tags: docker-compose, healthcheck, restart | Confidence: 0.65

Error Pattern:

unhealthy|restarting.*container

Root Cause:

The health check hits an endpoint before the application has finished starting (no ``start_period``), or checks a path that does not exist, or the app is crashing immediately on boot due to a missing environment variable and the health check is correctly reporting that.

Fix:

Confirm the app starts cleanly with ``docker logs``, add a ``start_period`` to the health check to allow for startup time, and double-check the health check path matches an endpoint that actually exists.

=== END E030 ===

=== ENTRY E031 ===

Railway: application failed the health check and was rolled back

Language: General | Framework: Railway | Tags: railway, deployment, port | Confidence: 0.88

Error Pattern:

health check failed|failed to respond

Root Cause:

The application is listening on a hard-coded port instead of the ` \$PORT ` Railway injects at runtime, so the platform's health check cannot reach it even though the app is running correctly.

Fix:

Bind the server to `0.0.0.0:\$PORT` using the environment variable, never a literal port number, in both the Dockerfile CMD and any code that constructs the listen address.

Patch Template:

```
--- a/Dockerfile
+++ b/Dockerfile
@@
-CMD ["uvicorn", "app.main:app", "--port", "8000"]
+CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

=== END E031 ===

=== ENTRY E032 ===

Git: 'fatal: refusing to merge unrelated histories'

Language: General | Framework: Git | Tags: git, merge | Confidence: 0.80

Error Pattern:

refusing to merge unrelated histories

Root Cause:

Two branches (often a freshly initialised local repo and an existing remote one) do not share a common commit ancestor, so Git refuses the merge as a safety measure against accidentally combining unrelated projects.

Fix:

If the histories genuinely should be combined, pull with `--allow-unrelated-histories`; if not, the correct fix is to clone the remote instead of re-initialising a new local repository.

=== END E032 ===
